Day 55 – API Management (APIM) – Part 4: Response Cache & Basic Auth Policy

Class date: 12th September 2026 | Module: SAP API Management – Caching Policies (continued) & Security Policies

All accurate again – verified against SAP's own Integration Suite documentation and Apigee policy references. The Response Cache vs. Populate Cache distinction, Invalidate Cache's purpose, and the Basic Authentication policy's Encode/Decode mechanics all matched official documentation precisely.

1. Recap (start here first)

Spike Arrest and **Cache Refresh** – once all the policies are configured, a full demo will be shown.

2. Response Cache Requirement

Quick recap of the two caching policies covered so far:

- **Populate Cache** – write the data to the cache.
- **Lookup Cache** – get the data from the cache.

Now – Response Cache vs. Populate Cache, what's the real difference?

Policy	What it stores
Populate Cache	Stores **only 1 variable** – e.g. a **token.**
Response Cache	Stores the **complete payload** – this is important to remember.

Example requirement used in class

Source → **Target**, keyed on something like **Customer ID** or **Order ID.**

Target system sample data (Orders):

OrderID	OrderName	Status
1	Order1	In Progress
2	Order2	In Progress
3	Order3	Delivered

3. Real-time scenario – why Response Cache matters

Setup: in the Target system, data only **updates once, on the 1st of every month.** If the Source keeps calling the Target repeatedly, **before the next month's 1st date**, what actually happens?

Answer: the **same data will be retrieved** every single time, right up **until the next month's 1st date** – because nothing has actually changed on the Target side in between.

The solution:

- Every month, on the **1st**, the Source system calls the Target endpoint, gets the **complete required payload**, and **stores it in a Response Cache.**

- From the **next call onward**, the data is served **from the Cache (Response Cache)**, **not** from the Target system.

This is exactly the use case for the Response Cache policy in APIM – avoiding unnecessary repeat calls to a backend whose data simply hasn't changed.

4. Response Cache – limitation

Response Cache's usefulness depends entirely on **how often the Target system's data actually changes:**

- **Monthly updates, weekly updates** (data does **not** update frequently) → Response Cache works **well.**
- **Frequent updates** → Response Cache **does not help (X)** – if the Target's data keeps changing constantly, a cached response quickly goes stale, defeating the whole purpose.

5. The 4 Caching Policies, side by side

Policy	Purpose
Populate Cache	Token storing – writes a single variable (e.g. an OAuth token) into the cache.
Lookup Cache	Reads the already-stored token/variable back out of the cache.
Response Cache	Stores a **complete payload** – specifically to reduce the number of requests made to the Target. Data updates on a monthly/weekly cadence (i.e. **batch-style updates**) are the only scenario where this is genuinely useful.
Invalidate Cache	Clears the cache memory, due to some reason (e.g. data was manually corrected, or the cache needs to be forced fresh).

Lookup Cache logic: does the token already exist in the cache?

- **Yes** → skip calling the Target for a new token; use the cached one.
- **No** → proceed to request a fresh token from the Target.

This Yes/No check is configured using a **Condition String** on the Lookup Cache policy.

With this, Traffic Management Policies are closed out – the full set covered across these classes: **Access Control, Quota, Spike Arrest, Reset Quota, Populate Cache, Lookup Cache, Response Cache, Invalidate Cache.**

6. Now, Security Policies

Security Policies cover everything related to **Authentication** and **Restrictions** – this is the category we move into next.

(1) Basic Auth Policy

The **Basic Auth policy** has **two operations: Encode** and **Decode.**

Operation	What it does	Direction

Decode	Takes the Basic Auth value already present in the headers, and **decodes** it back into the raw **Username & Password.**	**From** the Source (reading credentials the Source sent in).
Encode	Takes a **Username & Password**, and **encodes** them into the Basic Auth header value.	**To** the Target (preparing credentials to send onward).

If you want to extract the user credentials from the Authorization header, you use the Decode operation – it reads the incoming **Authorization** header and gets you the **Username and Password** out of it.

Why decode at all? When the Source system calls in with Basic Auth credentials sitting inside its Authorization header, APIM needs the raw Username/Password values out of that header (for example, to validate them, log them, or re-map them) – **Decode** is how that extraction happens.

Why encode at all? Before calling the **Target**, if the Target itself expects an **Authorization header** → **Basic Auth**, APIM must build that header. The **Encode** operation takes the Target's own Username and Password and combines them into the correctly-formatted header value – concatenated with a colon and **Base64**-encoded (e.g. **Base64(username:password)**).

Example – Postman demo

```
Postman -> Authorization tab -> Type: Basic -> enter Username, Password -> sent to
-> CPI -> CPI verifies the credentials
```

Quick-recall summary

- **Populate/Lookup Cache** = single variable (e.g. a token); **Response Cache** = the full payload, used to avoid repeat calls to a Target whose data barely changes (monthly/weekly updates only – not useful for frequently-changing data).
- **Invalidate Cache** = clears cache memory on demand.
- Traffic Management Policies, full set: **Access Control, Quota, Spike Arrest, Reset Quota, Populate/Lookup/Response/Invalidate Cache.**
- **Security Policies** start with **Basic Auth**: **Decode** (extract Username/Password from an incoming Authorization header, from the Source) vs. **Encode** (build a Base64 Basic Auth header to send onward, to the Target).